

TECMILENIO

Gestión y Automatización de Pruebas de Software

FinLab Eats

Portafolio técnico y reporte final

Prototipo funcional · Pruebas automatizadas · Integración continua

Equipo

Integrante	Matrícula
Stephani Alexandra Quintanilla Cervantes	2976101
Alan Gael Quintanilla Clemente	3106716
Gerardo Issael Puente Salazar	03109128

Fecha del cierre técnico: 24 de septiembre de 2026.
Edición consolidada con las capturas aportadas por el equipo.

Resumen ejecutivo

FinLab Eats demuestra un flujo de registro, consulta de restaurantes, selección de productos, creación de orden y registro de pago simulado. La validación final ejecutó pruebas funcionales con Playwright, pruebas de API con Newman y una prueba de rendimiento con k6. GitHub Actions automatiza parte de las pruebas de API y conserva reportes y logs.

La validación técnica EDT 5.1 está documentada como completada. Se mantienen tres fallas de validación HTTP 400→500, una debilidad de autorización e integridad referencial y 160 hallazgos estáticos de Helm. El cierre técnico no equivale a una aprobación para producción ni a la aprobación de todas las pruebas.

Esta edición integra seis capturas originales, procedimientos de reproducción, trazabilidad de la hipótesis, resultados, decisiones, quinta retrospectiva y una matriz de cumplimiento. La actualización administrativa de Trello y la hipótesis explícita en el README siguen pendientes de acreditación.

Versión de referencia: chore/static-iac-review
Commit: a3bfdd5833184230a9317b61e766fe5aab767038

1 Introducción, objetivo e hipótesis

Caso y problemática inicial

El caso de estudio es una plataforma de reparto de comida inspirada en el flujo de DiDi Food. El avance del proyecto partió de un fallo de despliegue con Terraform y Helm, identificado como “context deadline exceeded”. La problemática se concentró en detectar a tiempo configuraciones incorrectas, dependencias no disponibles y fallas de comunicación entre servicios.

Los escenarios de pods, conexiones y órdenes del avance se utilizaron para diseñar las pruebas. Los ejemplos hipotéticos de ese documento no se contabilizan como ejecuciones observadas. El cierre se sustenta en los logs publicados y las capturas incorporadas en este portafolio.

Objetivo general

Mejorar la calidad y estabilidad del backend mediante una estrategia de pruebas de software que permita detectar de forma temprana fallas en el despliegue, disponibilidad e integración de los servicios, reduciendo el riesgo de liberar versiones con componentes no funcionales o errores que afecten los procesos principales de la aplicación.

Hipótesis de trabajo

Si se incorporan pruebas estáticas, smoke, de integración y de regresión al proceso iterativo del prototipo, con criterios verificables y evidencias conservadas, será posible identificar defectos de configuración y comunicación antes del cierre y reducir el riesgo de liberar componentes no funcionales. La reducción del tiempo de diagnóstico y de despliegues fallidos requiere mediciones comparables para comprobarse cuantitativamente.

Esta formulación consolida la hipótesis del avance. Se conserva el caso y el objetivo presentados previamente; no se dispone en los materiales revisados de un acta independiente de aprobación docente.

Alcance y evolución del caso

El MVP incluye frontend React, gateway, servicios de usuarios, órdenes y pagos, y cuatro bases PostgreSQL. El catálogo de restaurantes y menú usado por las pruebas está definido en memoria en el gateway. El inicio de sesión de la interfaz es simulado y el pago representa un registro interno del prototipo.

La validación final se ejecutó sobre Docker Compose aislado. Terraform y Helm conservaron su función de infraestructura como código y fueron revisados estáticamente. No se ejecutó un despliegue del chart en un clúster real durante esta validación; por ello, la evidencia no permite afirmar que se resolvió integralmente el fallo original de Kubernetes.

Criterio de cierre

Se considera ejecutada la validación cuando existen resultados reproducibles del MVP, tres tipos de pruebas, una corrida real de CI, evidencias y riesgos registrados. Los defectos pendientes deben permanecer visibles. El cierre administrativo y la entrega académica se revisan por separado en la sección 12.

2 Organización y diseño técnico

Roles y cronograma del avance

Responsable	Función principal
Alexa — QA Lead / Project Manager	Estrategia, riesgos, métricas, seguimiento y organización del portafolio.
Gerardo — DevOps QA	Infraestructura, despliegues, salud, pruebas smoke y recolección de logs.
Alan — QA / Automatización	Casos, integración del backend, automatización y reejecución.

Fase	Días	Esfuerzo	Dependencia
1 Diagnóstico	2	8 h	Ninguna
2 Diseño de pruebas	3	12 h	Fase 1
3 Ejecución y automatización	5	23 h	Fase 2
4 Corrección y reejecución	3	12 h	Fase 3
5 Cierre	2	7 h	Fase 4
Total estimado	15	62 h	Secuencia de cinco fases

El plan comprende 22 actividades EDT y 15 días hábiles, equivalentes a tres semanas de trabajo. Las 62 horas son esfuerzo acumulado estimado del equipo; no se presentan como horas reales registradas ni como calendario efectivamente cumplido. Fuente: README de la versión de referencia.

Arquitectura y decisiones

Componente / decisión	Responsabilidad y justificación
Frontend React	Presenta registro simulado, restaurantes, menú, carrito y órdenes. La captura funcional acredita el estado visible de una orden pagada.
Gateway / backend	Expone /api, enruta solicitudes y consulta su propia PostgreSQL. Centraliza la integración; su manejo de errores constituye un defecto abierto.
Usuarios, órdenes y pagos	Tres servicios con bases separadas. Permiten probar comunicación y persistencia por componente; requieren validaciones entre entidades.
Compose y volúmenes aislados	Permiten reconstruir el entorno de validación sin modificar contenedores ni datos preexistentes.
Terraform, Helm y Checkov	Mantienen el diseño declarativo y detectan configuraciones riesgosas. El análisis estático no sustituye la ejecución del clúster.

3 Trazabilidad y estrategia de validación

La selección de pruebas responde a los riesgos del caso. Se combinó validación de interfaz, contratos API, disponibilidad, rendimiento básico e infraestructura. Cada resultado se interpreta dentro de su entorno y alcance.

Riesgo / hipótesis	Prueba y evidencia	Resultado y mejora
Detectar indisponibilidad antes del uso	Compose, /healthz, /readyz, pg_isready; informe EDT 5.1.	Servicios y cuatro bases disponibles. Ampliar readiness a dependencias críticas.
Evitar regresiones visibles	Playwright: carga de la aplicación y registro, menú y carrito; figura 2.	2/2 aprobadas. Extender E2E a checkout y agregar selectores estables.
Validar integración y persistencia	Newman: crear usuario, orden y pago; consultas posteriores; figura 5.	Flujo principal satisfactorio. Incorporar control de identidad y relaciones.
Identificar entradas inválidas	Newman: campos obligatorios ausentes; figura 6.	3 fallas 400→500. Corregir propagación segura de errores del gateway.
Detectar degradación básica	k6, 10 usuarios virtuales durante 15 s; figura 3.	150 solicitudes, 0 % errores. Ampliar duración y escenarios antes de inferir capacidad.
Prevenir configuración insegura	Checkov de Dockerfiles y Helm; reportes antes/después.	Docker: 10→0 hallazgos; Helm: 179→160. Tratar riesgos restantes y probar en clúster.
Automatizar comprobación repetible	GitHub Actions con Newman y artefactos; figura 4.	CI ejecutado. Hacer bloqueantes los negativos una vez corregidos.

Crterios y unidades de medición

Playwright informa casos de prueba; Newman informa solicitudes y aserciones; Checkov informa controles evaluados por recurso. Sus cantidades no se suman ni representan cobertura porcentual del software. La aprobación de una aserción que reproduce una vulnerabilidad no acredita una protección de seguridad.

La colección API completa conserva un criterio de rechazo de datos inválidos: HTTP 400. En CI, ese grupo se ejecuta sin bloquear el job para conservar visibilidad de defectos conocidos, con un reporte separado que permanece fallido.

Conclusión sobre la hipótesis

La evidencia apoya cualitativamente la detección de defectos y riesgos durante la iteración. No se registró una línea base controlada del tiempo de diagnóstico, la tasa de despliegues fallidos ni la tasa de reejecución. Esos beneficios esperados siguen sin cuantificación y no se atribuyen causalmente a las herramientas.

4 Prototipo y validación funcional

Entorno EDT 5.1

La ejecución del 24 de septiembre de 2026 utilizó el commit base 356ebbe9069cd504146a9e9165e5a1c513d9d6ac y el proyecto Compose finlab-edt51-356ebbe. Se asignaron puertos alternativos 8081 para frontend y 3100 para gateway, con cuatro volúmenes exclusivos. Los contenedores preexistentes no se detuvieron ni modificaron.

Comprobación	Resultado registrado
Stack aislado	9 contenedores activos: 5 aplicaciones y 4 PostgreSQL.
Salud de aplicaciones	Frontend, gateway, usuarios, órdenes y pagos: healthy.
Bases de datos	Las 4 respondieron a pg_isready. La base principal no tenía HEALTHCHECK propio; su disponibilidad se verificó expresamente.
Disponibilidad HTTP	Frontend /, gateway /healthz y /readyz: HTTP 200.
Operaciones funcionales	Restaurantes y menú: 200. Usuario, orden y pago: 201. Consultas posteriores de entidades: 200.
Comunicación interna	Los servicios de usuarios, órdenes y pagos respondieron desde el entorno interno.

Correcciones observadas en el proceso

El arranque inicial se vio afectado por la configuración de conexión del gateway y una condición de carrera en users-service. Se declaró DB_HOST=db en Compose, se incorporó el healthcheck de users-db y se añadieron reintentos al servicio de usuarios. La disponibilidad posterior se comprobó con solicitudes reales y pg_isready.

La ejecución con usuarios no root exigió corregir permisos de nginx sobre caché, configuración y /run. Su healthcheck se ajustó a 127.0.0.1. Los cuatro servicios backend se comprobaron con uid=1000 (node) y el frontend con uid=101 (nginx).

Evidencia funcional visible

La figura 1 muestra una orden con ID 4, total de \$80.00 y estado pagado. Complementa los logs de creación y consulta de entidades. Acredita el comportamiento mostrado en la interfaz; el pago y el inicio de sesión son simulados y no demuestran una transacción bancaria ni autenticación robusta.

Límites de disponibilidad

/readyz consulta la disponibilidad del gateway y su base de datos. No verifica por sí solo toda la cadena de usuarios, órdenes y pagos. Tampoco se realizaron en esta fase pruebas de recuperación ante caída, failover ni despliegue real de Helm.

5 Automatizaciones funcionales y de API

Playwright — interfaz

Archivo: tests/e2e/app.spec.js. Los selectores anteriores se actualizaron a la interfaz real de FinLab Eats. El primer caso verifica la marca y el encabezado de inicio de sesión. El segundo registra un usuario con datos únicos, consulta Taquería Don Pepe, abre el menú y agrega Tacos al pastor al carrito, comprobando cantidad uno.

La última captura local muestra 2/2 pruebas aprobadas en 2.5 s, con un worker (figura 2). El registro EDT 5.1 conservó otra ejecución de 2/2 en 2.4 s. Son corridas diferentes. Estos dos casos no incluyen confirmar el pago; el checkout se acredita por el flujo API y la captura funcional.

Los selectores combinan roles, textos y clases CSS. El catálogo en memoria aporta datos estables, pero un cambio de interfaz puede exigir mantenimiento. Se propone incorporar identificadores de prueba y añadir un escenario E2E de orden y pago.

Postman / Newman — contratos e integración

La colección contiene 14 solicitudes y 26 aserciones. Usa el entorno FinLab Eats - Local, conserva identificadores entre solicitudes y consulta las entidades después de crearlas. Está organizada en cuatro carpetas:

Carpeta	Qué comprueba	Aserciones
1 Smoke	Disponibilidad inicial del backend.	2
2 Flujo principal (integracion)	Restaurantes, menú, usuario, orden, pago y consultas posteriores.	18
3 Negativos (error guessing)	Solicitudes inválidas y resultados esperados del contrato.	4
4 Seguridad (hallazgo real)	Reproduce aceptación de una orden para userId inexistente.	2

Ejecución	Solicitudes	Aprobadas / total	Salida
Colección completa local	14	23/26	1 — fallida
Grupo estable en CI	10	22/22	0 — aprobado
Defectos conocidos en CI	4	1/4	1 — fallido

Las tres fallas esperan HTTP 400 y reciben 500 al crear usuario, orden o pago sin campos obligatorios. Los servicios internos validan los datos, pero el gateway convierte sus errores en respuestas genéricas. La figura 6 conserva las fallas completas.

El grupo llamado “estable” incluye dos aserciones que reproducen un problema de seguridad. Su resultado 22/22 significa que las expectativas programadas se cumplieron; no significa que la aplicación haya aprobado una evaluación integral de seguridad.

6 Rendimiento y análisis de métricas

Diseño de la prueba k6

Archivo: tests/perf/smoke.js. Ejecuta 10 usuarios virtuales durante 15 segundos contra /api/items, usando PERF_BASE_URL o http://localhost:3000. Cada iteración comprueba HTTP 200 y espera un segundo. Los umbrales son p95 de duración HTTP menor a 1200 ms y tasa de errores HTTP menor al 1 %.

Métrica	EDT 5.1 aislado	Nueva captura local
Solicitudes / comprobaciones exitosas	150 / 150	150 / 150
Errores HTTP	0 %	0 %
Tiempo promedio	9.54 ms	8.01 ms
Mediana	No consignada en el resumen	2.68 ms
Percentil 95 (p95)	62.42 ms	68.69 ms
Máximo	66 ms	70.82 ms
Usuarios virtuales / duración configurada	10 / 15 s	10 / 15 s
Criterios de aceptación	Cumplidos	Cumplidos

La nueva captura (figura 3) corresponde a una reejecución local aportada por el equipo. El archivo de imagen no muestra fecha ni commit; se registra como evidencia complementaria y no se atribuye a la corrida aislada anterior. Sus resultados no sustituyen los logs históricos de EDT 5.1.

Interpretación

El p95 de 68.69 ms está por debajo del límite de 1200 ms y no hubo errores HTTP. Esto acredita que el endpoint probado cumplió los umbrales durante ese ensayo breve. El resultado no demuestra capacidad máxima, comportamiento bajo carga sostenida ni rendimiento de los flujos completos de órdenes y pagos.

Aunque el promedio de la nueva captura es menor, su p95 es mayor que el de EDT 5.1. Otra ejecución previa había registrado aproximadamente 42 ms. Sin condiciones controladas de máquina, carga de fondo, datos, calentamiento y versión, estos valores no permiten afirmar una mejora o degradación estable.

Ampliación propuesta

Registrar máquina, recursos asignados, commit y hora; repetir el mismo escenario con condiciones equivalentes; aumentar duración y concurrencia gradualmente; agregar creación y consulta de órdenes y pagos; medir saturación de CPU, memoria y base de datos. Definir umbrales por operación antes de ampliar el ensayo.

Métricas no disponibles

No se midieron cobertura de código, tasa de defectos por tamaño del software, disponibilidad prolongada ni reducción de tiempo de diagnóstico. El porcentaje 23/26 de Newman tampoco se utiliza como indicador global de calidad, porque mezcla verificaciones funcionales con reproducción de una debilidad de seguridad.

7 Integración continua y evidencia remota

El workflow `.github/workflows/api-tests.yml` ejecuta automáticamente una parte del conjunto de pruebas en GitHub Actions. Se activa por push a `main`, `fix/**`, `ci/**` y `chore/**`, por pull request hacia `main` y mediante `workflow_dispatch`. El pipeline implementa CI con un entorno temporal de pruebas; no publica una versión en producción.

Secuencia automatizada

1. Obtiene el repositorio y construye el stack definido por Docker Compose en `ubuntu-latest`.
2. Espera hasta 30 intentos de `/readyz`, con pausas de 2 segundos; si el backend no está listo, captura logs y falla.
3. Instala Newman 6.2.2 y ejecuta el grupo estable de 22 aserciones como condición bloqueante.
4. Ejecuta cuatro aserciones de defectos conocidos con `continue-on-error` únicamente en ese paso; conserva su resultado fallido.
5. Publica un resumen de ambos grupos, dos JUnit, `compose-ps.txt` y `compose.log` como artefactos.
6. Ejecuta `docker compose down` al finalizar para retirar el stack temporal.

Ejecución	Versión y resultado	Evidencia
36050549447	350e7b6 · success Estables 22/22; negativos 1/4.	Artefacto 10830142527 y copia verificada en <code>evidence/edt-5.1/</code> .
36051290279	a3bfdd5 · success · 54 s Estables 22/22; negativos 1/4.	Artefacto 10830018865. Capturas del resumen y logs: figuras 4–6.

Interpretación del estado verde

Una corrida anterior había terminado en verde por el uso general de `continue-on-error`, aunque Newman contenía tres fallas. El workflow se corrigió, pasó `actionlint` y se ejecutó remotamente. La versión actual distingue las expectativas estables de los defectos conocidos y conserva su JUnit fallido.

El estado `success` acredita que se cumplieron las condiciones bloqueantes del workflow. Las tres fallas 400→500 permanecen abiertas. La prueba que reproduce aceptación de `userId` inexistente sigue dentro del grupo estable; cuando se corrija el control, esa prueba deberá exigir el rechazo correspondiente.

Conservación y mantenimiento

Los artefactos deben descargarse o conservarse en el portafolio antes de su expiración según la retención del repositorio. El índice de evidencia conserva una copia de la ejecución 36050549447. Se registraron advertencias de GitHub sobre Node.js 20 en `checkout@v4` y `upload-artifact@v4`; se propone revisar versiones compatibles y reejecutar el pipeline tras actualizarlas.

8 Revisión estática de infraestructura

Checkov se aplicó a los cinco Dockerfiles, al chart Helm renderizado como manifiestos Kubernetes y a la configuración Terraform. Los reportes están en `infra/checkov-reports/`. Este análisis complementa los tres tipos de pruebas exigidos; no se presenta como una prueba DAST ni como validación de un clúster en ejecución.

Componente	Antes: pasa / falla	Después: pasa / falla	Interpretación
Helm renderizado	632 / 179	651 / 160	19 hallazgos CKV_K8S_21 eliminados; 21 IDs distintos permanecen.
5 Dockerfiles	154 / 10	186 / 0	Se incorporaron HEALTHCHECK y usuario no root.
Terraform	0 aplicables	0 aplicables	Ausencia de cobertura aplicable; no equivale a seguridad aprobada.

Cambios y verificaciones

Se incorporaron namespaces en los manifiestos pendientes, incluidos los HPA. El render con `--namespace finlab` produjo 26 recursos y los 26 declararon ese namespace. `helm lint` terminó sin fallos. El análisis posterior no conservó hallazgos CKV_K8S_21.

Los cinco Dockerfiles fallaban inicialmente los controles de HEALTHCHECK y USER. Después de las correcciones se obtuvieron cero hallazgos en ese escaneo. El total de controles evaluados aumentó porque las nuevas instrucciones activaron comprobaciones adicionales; por ello, 154 y 186 no representan una misma muestra fija.

`docker compose config --quiet`, `bash -n scripts/03_deploy_helm.sh` y `terraform validate` resultaron satisfactorios. Terraform se validó en una copia temporal limpia. `terraform fmt -check` detectó una alineación preexistente en `terraform.tfvars` que quedó fuera del cambio. No se ejecutaron `helm install` ni `terraform apply` contra un clúster real.

Riesgo residual de Helm

Familia de controles pendientes	Tratamiento propuesto
Recursos y disponibilidad	Definir requests/limits y probes según mediciones y dependencias reales.
Identidad y aislamiento	Imponer securityContext, runAsNonRoot, seccomp y reducción de capacidades.
Imágenes y secretos	Fijar imágenes inmutables o digests; revisar política de descarga y gestión de secretos.
Red y permisos Kubernetes	Diseñar NetworkPolicies y desactivar tokens de ServiceAccount innecesarios.

Los 160 hallazgos están enumerados por ID, ocurrencias y recursos en `riesgo-aceptado.md`. La aceptación es temporal y se limita a desarrollo local. Caduca al pasar a un entorno compartido o incorporar datos y secretos reales. Checkov no reportó severidades en las salidas utilizadas; no se inventa una clasificación automática.

9 Defectos y mejoras propuestas

El cierre conserva resultados fallidos y riesgos abiertos. Las prioridades siguientes son propuestas del análisis del equipo, no severidades asignadas por Checkov. Los responsables corresponden a los roles del plan y deberán confirmar su programación en el tablero.

ID / prioridad	Hallazgo e impacto	Acción y criterio de cierre
DEF-API-001 Alta Alan	Tres entradas inválidas devuelven 500 en vez de 400. Se confunden validación y fallo de dependencia.	Propagar de forma controlada el estado y contenido permitidos de errores de validación; distinguir timeout y conexión fallida. Cerrar al aprobar los 3 negativos y nuevas regresiones.
SEC-API-001 Alta Alan	POST /api/orders acepta userId inexistente. No se acredita identidad ni propiedad de la orden.	Validar existencia del usuario, identidad y autorización en backend. Sustituir la expectativa que reproduce el defecto por rechazo verificable; probar acceso entre usuarios.
INF-001 Alta antes de promoción Gerardo	160 hallazgos Helm: aislamiento, recursos, imágenes, probes y red.	Priorizar los 21 IDs, justificar excepciones y reescanear. Validar el despliegue y políticas en un clúster antes de otro entorno.
COV-001 Media Gerardo	Terraform sin controles Checkov aplicables y sin ejecución real del despliegue.	Complementar con revisión específica, validación/plan controlados y pruebas de integración del despliegue.
OBS-001 Media Alan / Gerardo	Readiness del gateway no cubre servicios dependientes.	Definir señal de preparación integral o chequeos por componente y verificar comportamiento con dependencias caídas.
DOC-001 Antes de entregar Alexa / equipo	Trello sin actualización acreditada y README sin hipótesis explícita.	Actualizar tablero y README, conservar enlace o captura del estado final y añadir registro de revisión.

Orden recomendado de la siguiente iteración

Primero corregir contratos de error e identidad; después ampliar regresión de API y E2E, endurecer Helm y comprobar el despliegue real. Finalmente repetir mediciones bajo condiciones comparables. La decisión de no corregir todo durante el cierre mantiene la evidencia histórica; no elimina el impacto de los defectos.

Estado técnico

El MVP está disponible para demostración académica con límites declarados. EDT 5.1 está cerrada como actividad de validación ejecutada y documentada. La calidad del producto sigue siendo parcial: las pruebas detectaron defectos que requieren otra iteración.

10 Procedimientos de reproducción

Usar la rama chore/static-iac-review y registrar git rev-parse HEAD junto con fecha, herramientas y entorno. Los comandos siguientes están escritos para CMD de Windows, desde la raíz del repositorio salvo donde se indica. Docker Desktop debe estar iniciado. Las ejecuciones crean datos de prueba; conservarlos separados de datos reales.

Arranque y disponibilidad

```
docker compose up --build -d
docker compose ps
curl http://localhost:3000/healthz
curl http://localhost:3000/readyz
```

Para reproducir la captura local de interfaz en el puerto 5173, abrir otra terminal, entrar a apps\frontend, ejecutar npm ci y npm run dev. Confirmar la URL mostrada por Vite. El entorno aislado EDT 5.1 utilizó otros puertos y su procedimiento se conserva en el índice de evidencia.

Playwright desde su propio directorio

```
cd tests\e2e
npm ci
npx playwright install chromium
set "E2E_BASE_URL=http://localhost:5173"
npx playwright test app.spec.js
```

Ejecutar desde tests\e2e evita invocar una instalación distinta desde la raíz. Esta corrección resolvió en la sesión local el error "Playwright Test did not expect test() to be called here". E2E_BASE_URL debe coincidir con el frontend activo; la configuración predeterminada usa 8080.

Newman — colección completa

Volver a la raíz del repositorio. El carácter ^ continúa el comando en CMD. La colección completa conserva una salida 1 mientras existan los tres defectos de validación.

```
npx newman@6.2.2 run ^
  "tests/api/FinLab Eats - API Tests.postman_collection.json" ^
  -e "tests/api/FinLab Eats - Local.postman_environment.json" ^
  --reporters cli,junit --reporter-junit-export newman-full.xml
```

k6 — ensayo breve

```
k6 run -e PERF_BASE_URL=http://localhost:3000 tests\perf\smoke.js
```

Para la separación exacta de carpetas Newman, consultar el workflow enlazado. Conservar logs completos, códigos de salida, JUnit y resumen k6 junto con la captura. No sustituir evidencias previas por resultados de una nueva corrida sin identificar la versión.

Versiones registradas en EDT 5.1: Docker 29.8.0; Compose 5.5.1; Node 24.16.0; npm 11.13.0; Playwright 1.58.2; Newman 6.2.2; k6 2.2.0; Checkov 3.3.19; Helm 3.16.4; actionlint 1.7.7.

11 Quinta retrospectiva y cierre

¿Qué aprendizajes se obtuvieron al desarrollar el prototipo?

Aprendimos a distinguir señales de calidad con alcances diferentes. Un contenedor running no garantiza que su base esté lista; /readyz no cubre automáticamente todos los microservicios; un pipeline verde puede contener un paso no bloqueante fallido. La validación aislada, las solicitudes reales y la lectura de los JUnit permitieron evitar conclusiones basadas únicamente en indicadores visuales.

También aprendimos a conservar trazabilidad entre versión, entorno y resultados. Las dos ejecuciones de k6 de este informe tienen p95 distintos (62.42 y 68.69 ms), aunque ambas cumplen sus umbrales. La comparación requiere condiciones controladas. La hipótesis queda apoyada en detección de defectos, sin atribuir mejoras de tiempo que no medimos.

¿Qué técnicas y herramientas funcionaron mejor y por qué?

Newman permitió encadenar usuario, orden y pago, comprobar persistencia y repetir los tres errores 400→500. Playwright validó la experiencia visible desde el registro hasta el carrito. k6 aportó un ensayo breve con criterios explícitos. Estas herramientas se complementaron porque observan capas distintas del mismo prototipo.

Checkov ayudó a localizar configuraciones corregibles en Dockerfiles y namespaces, mientras que GitHub Actions hizo repetible la comprobación de API y conservó evidencias. Los logs y reportes resultaron esenciales para interpretar qué significaba aprobar una aserción o mantener un defecto conocido.

¿Qué problemas se presentaron y cómo se resolvieron?

El arranque requirió corregir la conexión del gateway, el healthcheck de users-db y los reintentos de users-service. El uso de usuarios no root obligó a ajustar permisos de nginx y su comprobación de salud. Las pruebas E2E se adaptaron a la interfaz real; para reejecutarlas localmente se utilizó el directorio tests/e2e y la URL correcta del frontend.

La primera configuración del pipeline ocultaba fallas con continue-on-error. Se corrigió la espera de readiness, se separaron los grupos y se publicaron JUnit y logs. Los tres errores de validación del gateway y la creación de órdenes con userId inexistente siguen abiertos; se documentan con propuestas y criterios de cierre en la sección 9.

Compromisos de mejora continua

En la siguiente iteración se priorizarán autorización y contratos de error, un E2E de checkout, readiness por dependencia y pruebas reales del chart. El equipo deberá definir responsables y fechas en Trello, repetir las pruebas afectadas y conservar tanto la evidencia fallida como la posterior corrección.

Conclusión

El portafolio demuestra un MVP funcional y la ejecución de tres tipos de pruebas, con automatización remota verificable. El cierre técnico registra resultados y limitaciones. La entrega administrativa requiere acreditar el tablero actualizado y completar la hipótesis del README; el producto conserva riesgos que impiden presentarlo como listo para producción.

12 Verificación contra la actividad y la rúbrica

Esta matriz revisa los materiales disponibles y la versión publicada. “Acreditado” indica evidencia identificable dentro del alcance; no anticipa la calificación docente. Los puntos pendientes no se presentan como completados.

Requisito / peso de rúbrica	Evidencia en este portafolio	Estado
MVP y al menos 3 tipos de pruebas — 20 %	Secciones 4–6; Playwright, Newman y k6; figuras 1–3, 5–6.	Acreditado; defectos abiertos declarados.
Portafolio técnico y evolución — 20 %	Caso, hipótesis, decisiones, trazabilidad, procedimientos, métricas, riesgos y evidencias enlazadas.	Acreditado en este documento.
Automatización CI/CD — 30 %	Workflow y dos corridas reales; JUnit y logs. Sección 7 y figuras 4–6.	Acreditado para CI; no despliegue productivo.
Repositorio, configuración, README y tablero — 20 %	Scripts en tests/, reportes en evidence/ e infra/, README con caso, objetivo, cronograma y enlaces.	Parcial: hipótesis no explícita en README y tablero final sin acreditar.
Quinta retrospectiva — 10 %	Sección 11: tres preguntas respondidas con problemas, decisiones y resultados concretos.	Incluida.
Caso, objetivo e hipótesis aprobados	Secciones 1–3 y avance aportado.	Documentados; acta independiente de aprobación no aportada.
Cronograma y roles	Sección 2 y README de la versión revisada.	Acreditado como planificación.
Reporte final PDF con evidencias y enlaces	Este PDF y su Word editable, con seis capturas originales y catálogo de fuentes.	Incluido.

Pendientes concretos de entrega

1. Trello: el último intento de mover EDT 5.1 a Done recibió 401 Unauthorized. Un integrante autenticado debe actualizar la tarjeta y revisar las actividades de documentación y cierre. Conservar una captura legible o enlace verificable del tablero final. No existe evidencia nueva que permita darlo por actualizado.
2. README: incorporar la hipótesis de la sección 1 o un resumen equivalente y enlazar este portafolio final. La revisión de la versión a3bfdd5 encontró objetivo, cronograma y enlaces, pero no una formulación explícita de la hipótesis.
3. Aprobación del avance: si la plataforma exige acreditarla por separado, adjuntar el registro docente existente. El documento conserva la información del avance sin fabricar una aprobación.

El archivo editable original del avance no fue aportado; se revisó su PDF. Esta entrega sí contiene un Word editable del portafolio final. Los pendientes externos anteriores no se resuelven únicamente al generar el documento.

13 Fuentes, enlaces y control de evidencia

Se contrastaron el avance en PDF, la rúbrica, el portafolio previo, el código y la documentación de la rama publicada, y las capturas aportadas. Los enlaces a archivos utilizan el commit de referencia para conservar trazabilidad; main puede no contener todavía estos cambios.

[Repositorio FinLab Eats](#)

[Versión revisada · a3bfdd5833184230a9317b61e766fe5aab767038](#)

[README · caso, EDT, cronograma, roles y enlaces](#)

[Informe de validación final EDT 5.1](#)

[Índice de evidencias EDT 5.1](#)

[Verificación conservada de la ejecución 36050549447](#)

[Workflow de pruebas API](#)

[GitHub Actions · ejecución 36050549447](#)

[GitHub Actions · ejecución final 36051290279](#)

[Artefacto final · JUnit y logs](#)

[Scripts de pruebas · E2E, API y rendimiento](#)

[Reportes Checkov y riesgos aceptados temporalmente](#)

[Tablero Trello · FinLab Eats Testing](#)

[Tarjeta EDT 5.1 · Ejecutar validación final](#)

Registro de versiones y fuentes documentales

356ebbe: base de la revisión estática y validación aislada.

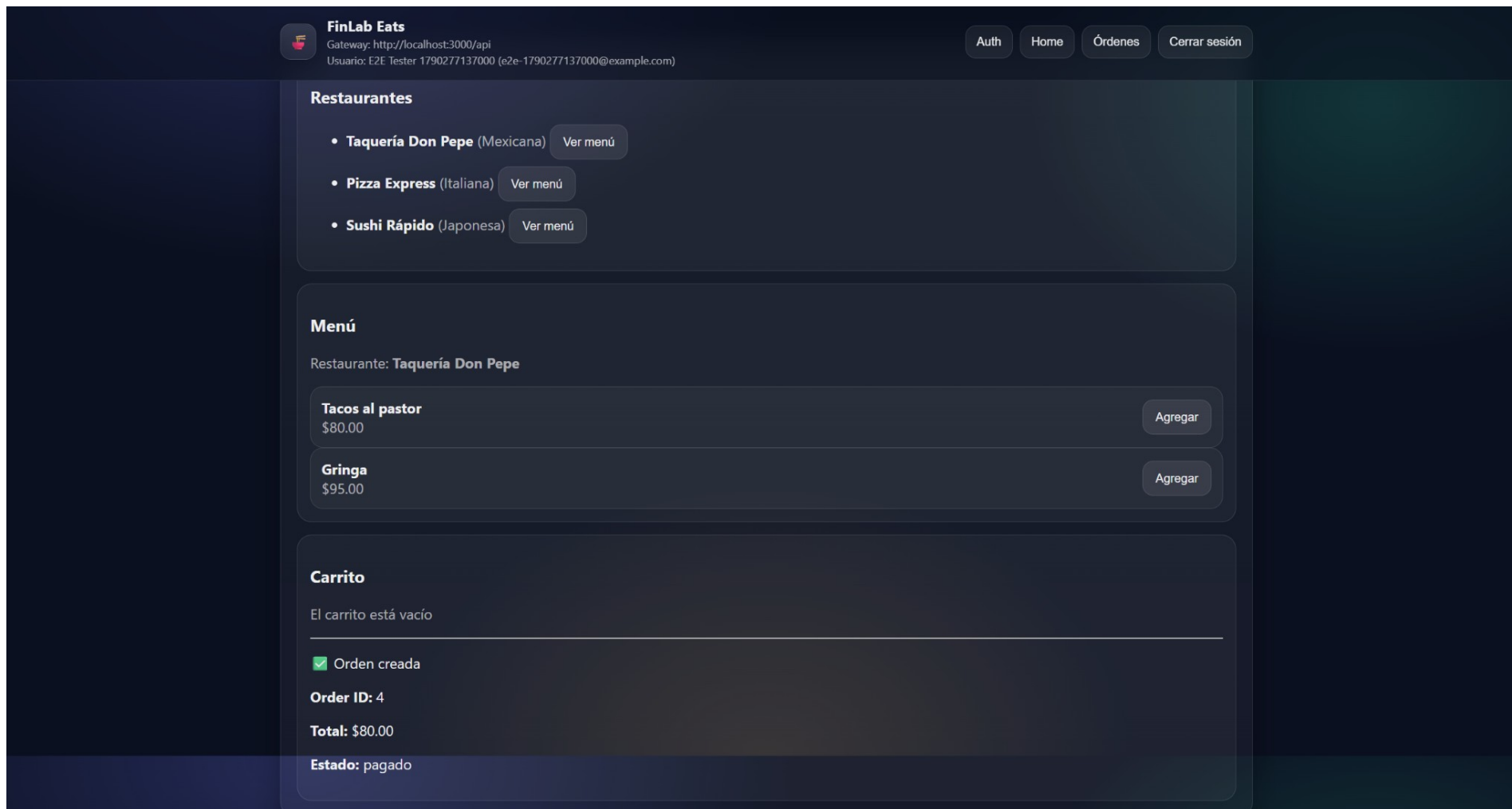
350e7b6: validación EDT 5.1 y endurecimiento del workflow.

a3bfdd5: cierre documental con evidencia de CI; referencia de esta revisión.

Materiales aportados: Avance_Proyecto_GAPS_NEW (1).pdf; Rubrica_Proyecto_Final.pdf; Proyecto_Final_Portafolio.docx; resúmenes técnicos del equipo y seis imágenes originales. Los reportes históricos mantienen sus cifras; las nuevas capturas locales se identifican por separado.

Los enlaces a artefactos o tableros pueden requerir acceso autorizado y están sujetos a disponibilidad o retención del servicio. Las capturas se incorporan como evidencia visual complementaria; los scripts, logs y reportes enlazados permiten revisar el detalle técnico.

Anexo visual • Figura 1 • Prototipo: orden registrada como pagada



La interfaz muestra Orden ID 4, total \$80.00 y estado pagado; el carrito quedó vacío después de confirmar. Evidencia manual del flujo visible del MVP. El pago y el acceso son simulados; no acreditan procesamiento bancario ni autenticación segura.

Fuente: captura original aportada por el equipo; resultados transcritos sin modificar la imagen.

Anexo visual • Figura 2 • Pruebas funcionales con Playwright

```
C:\Users\VladG\OneDrive\Desktop\didi-food-devops-alexa\tests\e2e>npx playwright test app.spec.js
Running 2 tests using 1 worker
ok 1 app.spec.js:3:1 › loads home and shows FinLab Eats branding (497ms)
ok 2 app.spec.js:15:1 › register, browse a restaurant menu and add a product to the cart (828ms)
2 passed (2.5s)
C:\Users\VladG\OneDrive\Desktop\didi-food-devops-alexa\tests\e2e>
```

Reejecución desde tests/e2e: 2 pruebas aprobadas en 2.5 s, un worker. Caso 1: carga y marca FinLab Eats. Caso 2: registro, menú y producto en el carrito. La captura no muestra fecha ni commit y se conserva como evidencia local complementaria.

Alcance acreditado por los dos escenarios

Escenario	Verificación	Límite
Carga de la página	Marca FinLab Eats y encabezado Iniciar sesión visibles.	No mide toda la disponibilidad de dependencias.
Registro, menú y carrito	Crea usuario, abre Taquería Don Pepe y comprueba Tacos al pastor con cantidad uno.	No ejecuta confirmar pago; se complementa con API y figura 1.

Fuente: captura original aportada por el equipo; resultados transcritos sin modificar la imagen.

Anexo visual • Figura 3 • Prueba breve de rendimiento con k6

```

/ ----- \ | | \ ----- /

execution: local
  script: tests\perf\smoke.js
  output: -

scenarios: (100.00%) 1 scenario, 10 max VUs, 45s max duration (incl. graceful stop):
  * default: 10 looping VUs for 15s (gracefulStop: 30s)

■ THRESHOLDS

http_req_duration
✓ 'p(95)<1200' p(95)=68.69ms

http_req_failed
✓ 'rate<0.01' rate=0.00%

■ TOTAL RESULTS

checks_total.....: 150      9.905792/s
checks_succeeded...: 100.00% 150 out of 150
checks_failed.....: 0.00%   0 out of 150

✓ status 200

HTTP
http_req_duration.....: avg=8.01ms min=507.8µs med=2.68ms max=70.82ms p(90)=9.36ms p(95)=68.69ms
{ expected_response:true }...: avg=8.01ms min=507.8µs med=2.68ms max=70.82ms p(90)=9.36ms p(95)=68.69ms
http_req_failed.....: 0.00% 0 out of 150
http_reqs.....: 150      9.905792/s

EXECUTION
iteration_duration.....: avg=1s      min=1s      med=1s      max=1.08s  p(90)=1.01s p(95)=1.08s
iterations.....: 150      9.905792/s
vus.....: 10      min=10      max=10
vus_max.....: 10      min=10      max=10

NETWORK
data_received.....: 40 kB 2.6 kB/s
data_sent.....: 12 kB 783 B/s

running (15.1s), 00/10 VUs, 150 complete and 0 interrupted iterations
default ✓ [=====] 10 VUs 15s

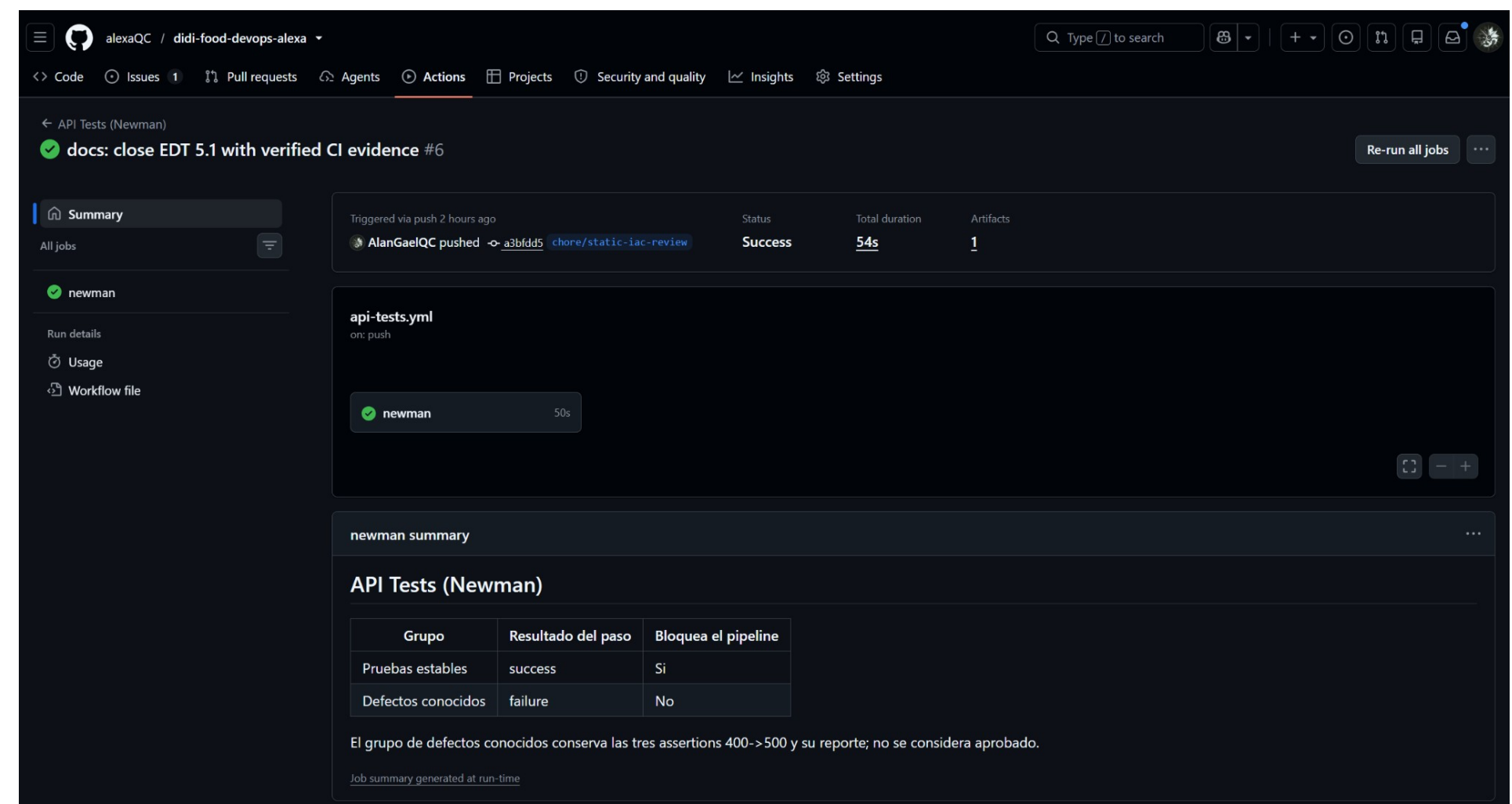
C:\Users\VladG\OneDrive\Desktop\didí-food-devops-alexa>

```

10 usuarios virtuales, 15 s configurados; 150 solicitudes y 150 comprobaciones satisfactorias; 0 % errores HTTP. Promedio 8.01 ms, p95 68.69 ms y máximo 70.82 ms. Ambos umbrales se cumplen. Corrida local distinta de EDT 5.1 (p95 62.42 ms).

Fuente: captura original aportada por el equipo; resultados transcritos sin modificar la imagen.

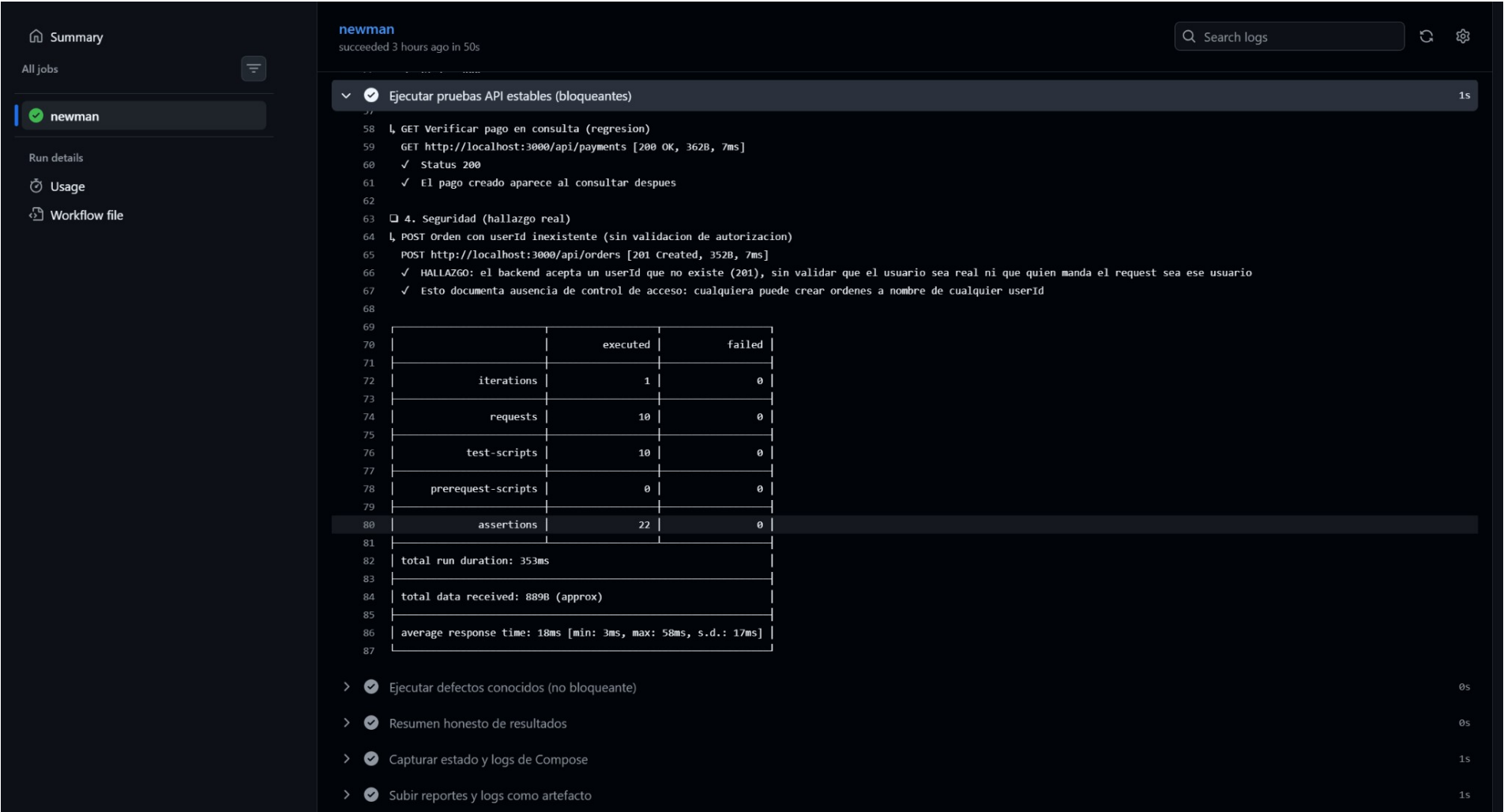
Anexo visual • Figura 4 • GitHub Actions: resumen de la ejecución final



Ejecución 36051290279, commit a3bfdd5, rama chore/static-iac-review: success en 54 s y un artefacto. El resumen distingue pruebas estables aprobadas de defectos conocidos fallidos y no bloqueantes; el estado global no implica aprobación de estos últimos.

Fuente: captura original aportada por el equipo; resultados transcritos sin modificar la imagen.

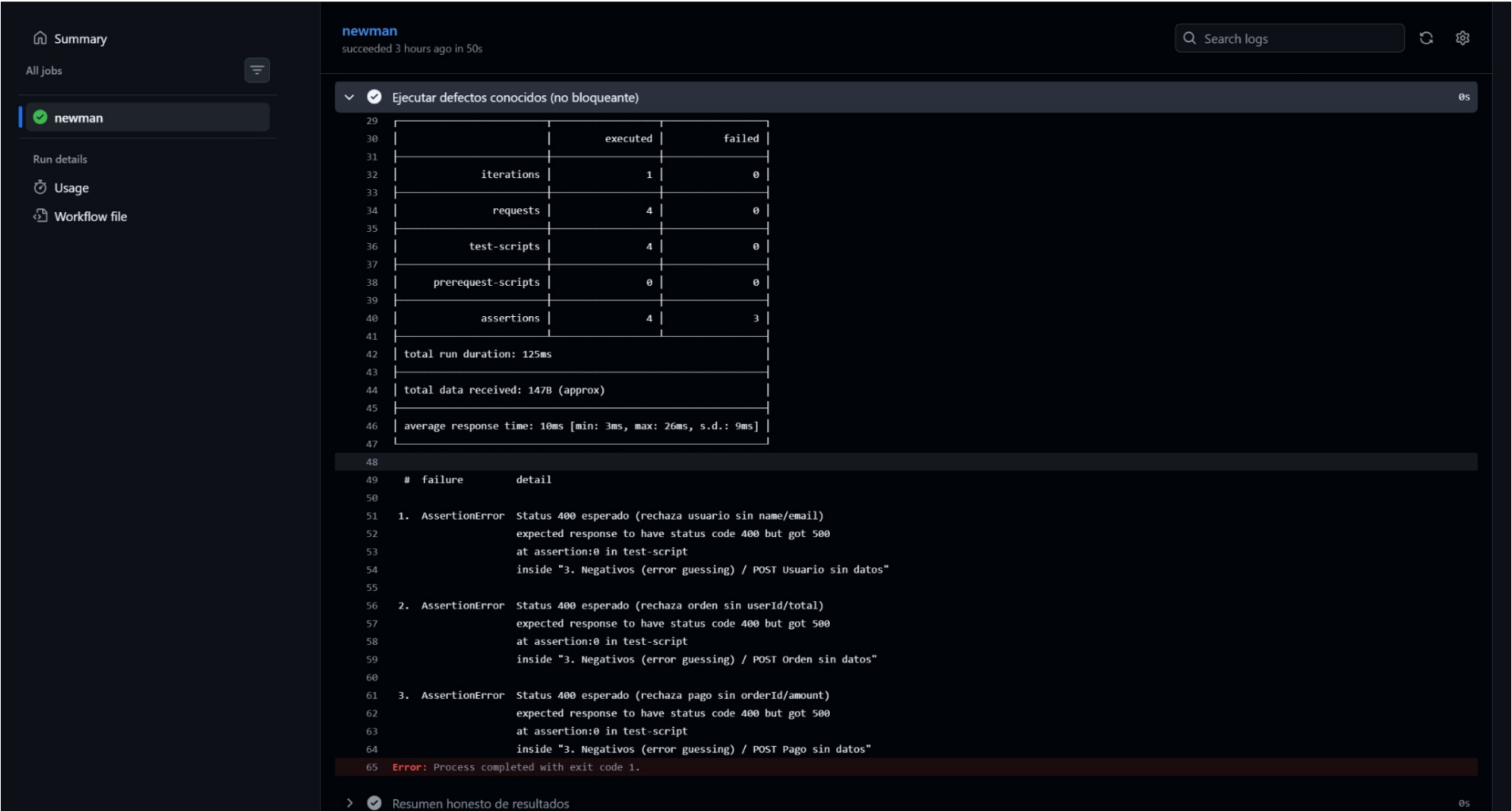
Anexo visual • Figura 5 • Newman: grupo estable y hallazgo de seguridad



10 solicitudes y 22 aserciones ejecutadas, cero fallas; duración 353 ms. La imagen también conserva la aceptación de una orden para userId inexistente. Las aserciones que reproducen esa debilidad pasan; no acreditan que exista control de acceso.

Fuente: captura original aportada por el equipo; resultados transcritos sin modificar la imagen.

Anexo visual • Figura 6 • Newman: defectos conocidos conservados como fallidos



4 solicitudes, 4 aserciones y 3 fallas; código de salida 1. Usuario sin datos, orden sin userId/total y pago sin orderId/amount esperan HTTP 400 y reciben 500. El paso es no bloqueante, pero sus resultados permanecen fallidos.

Fuente: captura original aportada por el equipo; resultados transcritos sin modificar la imagen.